Buffer Overflow Attacks in the Era of Gen-AI

Maxime Pierre Claude Reynaud

Student ID: 720028646

Abstract

In today's interconnected world, achieving complete software security is extremely challenging, as vulnerabilities continue to provide opportunities for exploitation. Buffer overflow attacks remain among the most enduring and impactful forms of software exploitation, enabling adversaries to manipulate program execution and gain unauthorized access. Although significant countermeasures such as stack canaries, Address Space Layout Randomisation (ASLR), and non-executable memory protections have been developed, buffer overflows persist due to legacy codebases, low-level programming practices, and the constant evolution of attack strategies. Recent advances in Generative Artificial Intelligence (Gen-AI) introduce a new dimension to defensive cybersecurity research and offers potential as a countermeasure through enhanced vulnerability detection, automated secure code generation, and the ability to adapt defensively to dynamic attack patterns. This project will specifically evaluate the effectiveness of Gen-AI models in guarding against modern buffer overflow variants, with a focus on recent vulnerabilities such as CVE-2025-6660 and CVE-2025-6191. By integrating literature review with targeted case studies, the research aims to assess the capacity of Gen-AI to strengthen existing defenses and to determine whether its adaptability can provide meaningful resilience against emerging buffer overflow exploitation techniques.

☑ **I have not used any GenAI tools in preparing this assessment.**

☑ **I certify that all material in this dissertation which is not my own has been identified.**

Signature: Maxime Pierre Claude Reynaud

1 Introduction

Before defining what buffer overflows are, how exploitation of this vulnerability is achieved, and why it remains a critical vulnerability decades after it was first documented, the reader needs to initially understand cybersecurity. It is a broader concept that will help gain a better grasp of what are buffer overflows. A possible definition of cybersecurity, proposed by IBM and one that can lay the foundation for a better understanding of the concept, is: "the practice of protecting people, systems, and data from cyberattacks by using various technologies, processes, and policies" [1].As observed by *Admass et al. (2024)* with the explosive growth of the Internet, digital assets and people are more exposed to cyber risks than ever before [2]. The general population might not realize but all online data, accounts, habits, medical records and more, everything relies on cybersecurity for staying private and not being accessible to people with malicious intent.

Studies have been attempting to detect and remediate increasingly complex vulnerabilities for decades since their discovery. Several papers note that although substantial evolution has occurred in tools and methodologies, the field is still considered immature [3], and as highlighted by de *Nobrega et al. (2024)*, attackers often remain a step ahead of the latest remediations [4]. Among the wide spectrum of existing software vulnerabilities, buffer overflows exist and persist [5]. First described and studied in 1972 by *Anderson* [6], this type of vulnerability occurs when the amount of data in the buffer exceeds its storage capacity and overflows into adjacent memory locations overwriting the initial data [5]. This class of vulnerability is inherent to certain programming languages that allow users to manipulate memory, control allocations, and provide limited bounds checking: C, C++, Fortran, and Assembly are vulnerable languages by default [7]. Today, buffer overflows and out-of-bounds memory operations are regarded as some of the most dangerous software weaknesses by the industry-standard Common Weakness Enumeration (CWE) framework [8].

Today, Artificial Intelligence (AI), and more precisely Generative AI (Gen-AI) adoption has accelerated despite limited user understanding of what it is and how it works. As stated by *Hikmawati and Mohammad (2025)* [9], users engage with a Gen-AI model when using ChatGPT, and this interaction has seen a surge of interest in recent years. Gen-AI is regarded by IBM as "AI that can create original content such as text, images, video, audio, or software code in response to a user's prompt or request" [10]. It is stated by OWASP that buffer overflows often emerge from inexact memory management due to poor assumptions about the size of data [7]. This can be exploited to read or write to unintended memory locations.

Existing literature has tried to examine how the buffer-overflow family of vulnerabilities is exploited, how it can critically affect users and organizations, and how, with the rise of generative AI (Gen-AI), researchers may be able to mitigate and identify these vulnerabilities before they are introduced in the wild and cause real harm. Our review will follow a four-section structure and will compare existing literature. Firstly, we will provide an overview of buffer overflow vulnerabilities, their mechanisms, and their continued relevance in modern computing systems. The second section will examine existing defense techniques and mitigation strategies, supported by relevant academic and industry research. Finally, we will introduce the role of Artificial Intelligence (AI) in cybersecurity, with a particular emphasis on Generative AI (Gen-AI) and its applications in secure code analysis.

2 The Buffer Overflow Vulnerability

2.1 Definition and Historical Context

Some might date the first appearance of a study describing buffer overflows to 1972, in the *Computer Security Technology Planning Study*, where it was described as improper function code address

validation leading to possible user overwrite [6]. Later came the much more documented case of a buffer overflow vulnerability that propagated through mail services and made the news by crashing computers through constant reinfection. "The Morris Worm," dating from 1988, is considered one of the oldest computer worms spread using the Internet [11]. What is described here as a computer worm has been defined as "a malicious code that spreads through Internet connections or a local area network (LAN)." The Morris Worm code made headlines and is reported to have spread to thousands of machines, copying itself 14% of the time without checking whether the computer was already infected. Another event that brought buffer overflows to public attention and was a turning point for early researchers was the 2001 "Code Red Worm" , documented in *Code-Red: a case study on the spread and victims of an internet worm* [12], it was reported to take control of Microsoft IIS web servers and in total had infected more than 350,000 hosts.

There is often a talk about the stack buffer overflow vulnerability being the biggest concern, although the truth is that there are a plethora of attacks involving writing to memory outside the intended boundaries of a program in a compiled language. According to Cloudflare [13] the earliest and most common type of buffer overflow is the "stack buffer overflow." The Computer Worm Classification [14], considers the Code Red stack buffer overflow exploiting worm that was previously mentioned as among the top ten most destructive worms in 2010. Later on came "heap buffer overflows," which targeted a different part of memory but were no less dangerous.

2.2 Mechanisms of Exploitation

Now that introduction to the concepts of stack and heap has been done, we need to take a deeper dive into how memory is managed and how it is possible to write to locations to which the user should not have access.

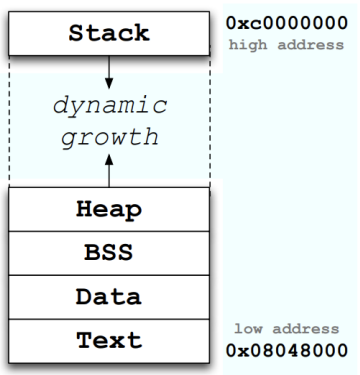

Figure 1: *Process layout in memory[15]*

In the above *Figure 1* taken from the study *On the evolution of buffer overflows* [15] observation of the memory layout of a compiled C binary can be made, in which we can distinguish the text segment, initialized data segment, and uninitialized data (BSS) segment sitting in the low addresses with a fixed size [16]. Higher up is the heap, which grows increasingly in addresses; this segment of memory is allocated at runtime using **calloc** or **malloc**. The stack, as described by *L. Ferres (2010)* [16], grows downward in addresses and is used to store local variables, pass arguments to functions, and hold the return address of the instruction to be executed after the function call returns. A typical buffer overflow would have an attacker fill a buffer with garbage, thereby overwriting memory beyond its bounds to alter return addresses or instructions in the code, or to read from locations the attacker should not have access to *(Littlejohn Shinder and Michael Cross, 2008)* [17]. Understanding that these actions can lead to privilege escalation and account takeover in certain cases proved by studies as *Stefan Nicula et al.* [18] is crutial.

Executable binaries often take the form of the *Portable Executable* (PE) format for Microsoft, or the *Executable and Linkable Format* (ELF) for UNIX-like operating systems [15]. A number of relevant tools are available for examining binaries statically and dynamically at runtime. Some of them include the runtime debugger *GDB (GNU Debugger)* initially made by *Richard Stallman in 1986* [19] and the Python library pwntools, which can be used to manipulate instructions. To decompile files and understand what the code is made of, tools like *Ghidra* [20] or *IDA* are industry standards in reverse engineering.

2.3 Continued Relevance in Modern Systems

Although extensively studied the lifespan of such a vulnerability continues for decades after its discovery, the existence of unchanged legacy code in organizational and operational platforms, combined with the inherent flaw of limited bounds checking in specific compiled languages like C and C++, encourages attackers to continue to pursue this vulnerability. Below is *Table 1* showcasing 8 of the 25 most dangerous software weaknesses in 2025 according to the Common Weakness Enumeration (CWE) framework [8].

CWE Ranking	CWE ID	CWE Name
1	CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
2	CWE-787	**Out-of-bounds Write**
3	CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
4	CWE-352	Cross-Site Request Forgery (CSRF)
5	CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
6	CWE-125	**Out-of-bounds Read**
7	CWE-78	Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
8	CWE-416	**Use After Free**

Table 1: 2024 CWE Top 25 Most Dangerous Software Weaknesses [8]

Out of the top eight weaknesses in *Table 1*, three of them are related to memory corruption and access violations. Included among these three is CWE-787, ranked number 2 in severity. This CWE is directly correlated with and leads to buffer overflows, demonstrating that memory exploitation is still regarded as critical in modern computing systems. A paper *Software Weaknesses, 2020* studying this CWE ranking for the year 2021 showed that CWE-787 was already first on the list back then [21]. With evolving systems, an increasingly interconnected world, and the rise of Internet of Things (IoT) devices and embedded systems, the risk of this vulnerability appearing on the Internet has increased due to their lack of basic security features as observed by multiple papers [22][23]. Recent Common Vulnerabilities and Exposures (CVE) reports from 2025 showcase vulnerabilities in specific versions of Google Chrome (CVE-2025-6660) [24] and PDF-XChange (CVE-2025-6191) [25] prove that this vulnerability is still very present and extremely damaging in the wild

3 Countermeasures and Defence Techniques

3.1 Compiler-Level and OS-Level Protections

To defend against buffer overflow vulnerabilities and secure code from memory manipulation, different types of protection mechanisms have been put in place. One of the most effective and ingenious is

Stack Smashing Protection (SSP), published by *Cowan et al.* in 1998 [26]. Their work introduced a small value at runtime called a *canary* on the stack before the return address. A *Red Hat* article written in 2022, gives precision that this value should be unknown to the attacker and is used to detect whether the control flow of a program has been hijacked [27]. If the value of the canary has been changed, it can be determined that there has been a stack memory overwrite and the program is terminated immediately as stated by *B. Bierbaume et al. (2018)* in *"Smashing the stack protec-tor for fun and profit"* [28]. Even if SSP is considered as one of the pillars of buffer overflow prevention techniques study showed that a bypass of it can be achieved with an information leak or by targeting other memory parts like the heap, although still considered a robust protection technique certain programing languages like *WebAssembly* are inherently proven to be vulnerable to buffer overflows and enable attacker to bypass SSP due to poor memory layout and runtime initialisation as demonstrated in *"Robust Stack Smashing Protection for WebAssembly"* [29].

Another very useful protection method is **Address Space Layout Randomization (ASLR)**, it is defined by *H. Shacham et al. (2004)* as a technique that works by randomly moving memory addresses of system components in the address space layout [30]. Nevertheless instead of patching the vulnerability, this technique focuses on making the exploitation of buffer overflows more challenging, keeping a door open for attackers accompanied with a trade-off in possible compatibility issues introduced by address randomization. It has been showed by *H. Marco-Gisbert et al. (2014)* that ASLR can still be rendered ineffective by exploiting an information leak or if a program remains resident in memory and thus retains the same randomized layout that can then be exploited [31].

The last method of this non-exhaustive list that will be considered is **Data Execution Prevention (DEP)** and the **No-Execute bit (NX)**. In a paper writen by *Stojanovski et al. (2007)* observation has been made that this protection method prevents the execution of injected code from non-executable memory regions such as the stack or heap [32] and relies on hardware support through the NX bit, which is used to mark data pages as non-executable as stated in *"The role of non-executable stacks in preventing buffer overflows"* [33]. While DEP works well against shellcode injection, *N. Stojanovski et al. (2007)*, demonstrated that it only prevents the execution of new code and that the control flow of a program can still be hijacked and methods like Return-Oriented Programming (ROP) are often used to bypass DEP [32].

All these methods are critical for safer programs when interacting with memory in unsafe manners or languages, but then arises the question of safe languages and their ability to manipulate memory as efficiently. If we take the example of **Rust**, it is said the the book *The Rustonomicon* [34] that "Rust can be thought of as a combination of two programming languages: Safe Rust and Unsafe Rust" meaning that for all the standard interaction you will be able to stay in the "Safe" language but for more fine grained, low level implementations you might need to start using the unsafe language. This paves a path for researchers to develop specific tools to cope with the enhances risk posed by the unsafe languages, for Rust *Hua et al.(2021)* [35], developed *Rupair* an automated rectification system used to detect and fix buffer overflows, considered as one of the most severe Rust vulnerabilities. In opposition *William Bugden and Ayman Alahmar* [36] consider that avoiding buffer overflows in Rust is trivial due to its inherent bound checking and rare use case of the unsafe language.

3.2 Analysis and Detection Tools

In addition to enhanced compiler and operating-system-level protections, the mitigation of buffer overflows can also be achieved by aiming for early discovery with static and dynamic analysis tools. As stated by *Discovering Buffer Overflow Vulnerabilities in the Wild: An Empirical Study (2014)* [37], most reporters typically use fuzzing for their discoveries and proceed with debugging tools to analyse the crash introduced by a fuzzer. The paper also states that static analysis tools are rarely used for buffer-overflow discoveries. Static analysis tools exist and are mostly used as general-purpose scanners

to detect not only buffer overflows aswell as other vulnerabilities (XSS, SQLi, CSRF, etc.); Semgrep and SonarQube are prime examples of multi-purpose static analysis tools. A 2004 study from MIT, *"Testing Static Analysis Tools Using Exploitable Buffer Overflows from Open Source Code"* [38], tested five frameworks and concluded that, generally, the detection rate had a low performance, and for the tools where it was not, they often reported a high number of false positives with 50% of findings for PolySpace and 43% for Splint.

Dynamic analysis tools, on the other hand, are much less common for buffer-overflow detection and have not been widely adopted for multiple reasons: "They cannot guard against all buffer overrun attacks, they break existing code, or incur too high an overhead," as stated by *Ruwase et al. (2004)* in *"A Practical Dynamic Buffer Overflow Detector"* [39]. Nevertheless, these tools still exist and can sometimes provide useful findings in terms of memory leakage and overwrite, for example, Valgrind and AddressSanitizer. This can be illustrated by a study conducted by NIST to evaluate the efficacy of certain tools on buffer-overflow detection, which proved that certain frameworks (here consideration of the *ARCHER* tool is made) have a very low false-positive rate and a 90% detection rate, yet are still prone to missing certain issues due to framework limitations *(Kratkiewicz et al., 2006)* [40]. Decompilers are also part of the process, in the sense that buffer overflows occur in compiled languages; to obtain a proper understanding of the code in human-readable form, researchers often have to return to the closest point to the initial readable code to find the source of the vulnerability. In this space, IDA Pro and Ghidra are the most popular tools on the market according to *LetsDefend (2025)* [41].

With the latest tools and knowledge, innovators are still faced with challenges and limitations regarding high numbers of false positives, high cost, etc. Even though the vulnerability is not recent, the exploitation patterns used are always evolving, and adaptability to these new techniques is a crucial part of any detection and mitigation tool. With all that said, this gap is not only an issue regarding buffer overflows but cybersecurity in general, and staying up to date with the latest exploits and trends has long been considered a Sisyphean task until very recently, with the growing public availability of AI and increasingly performant models.

4 The Role of Artificial Intelligence in Cybersecurity

4.1 AI for Secure Code and Vulnerability Mitigation

Since its creation in November 2022 [42], ChatGPT has revolutionized the way interaction with knowledge is done and it has seen its user base grow exponentially in that matter. The acronym GPT stands for *"Generative Pre-trained Transformer"*, and the generative part is considered a crucial component that differentiates this model from existing ones. In the Generative family of models we also find popular tools that include Claude and Gemini for content creation, DALL-E 2 an Midjourney for design and visual arts, Github and Microsoft copilot for coding and development [43]. The list of things we now employ Gen-AI for is in perpetual growth and new ways to employ these tools are discovered everyday, from electronics [44] to pharmaceutical education [45], the list extends and is evidence of a significant improvement of the numerical world. Artificial Intelligence (AI) is defined as a very broad term that encompasses all model types, with further classifications within it, including our core interest, Generative AI (Gen-AI) [46]. Research has already been conducted on this subject; using machine learning to detect source code vulnerabilities in C/C++ applications is a concept that has been studied by *Rajapaksha et al. (2023)* [47], where detection achieved a 0.96 F1 score for binary classification. Using convolutional neural networks for better pattern recognition and applying them to vulnerability detection, as described by *Amarasinghe et al. (2019)* [48] has also been a studied subject. All these techniques and models vary but no studies seem to have found a way to generalize protection and detection while still staying on top of the latest released exploits. *Schreiber et al. (2025)* [49] argues that using Gen-AI to augment human expertise might be the best way to address the topic instead of only relying on the model.

Gen-AI is used to create new content, unlike its peer machine-learning techniques, which mostly predict based on given datasets, as stated by *MIT* [50]. As surprising as it may seem, Gen-AI is observed as being used for vulnerability detection and remediation every day when code is pasted into ChatGPT and returned in a new, fixed, and upgraded version based on the initial request. The user initiating this might not even realize that their code was initially vulnerable, but it is now modified and fixed. It has been recognized by *Shukla and Shubham* [51] that Gen-AI has numerous applications in enterprise security and has to be considered when designing software, as described in *Table 2*. The ability to scan large codebases is also not to be neglected regarding possible time savings.

Advantages of Gen AI in Code Vulnerability Detection	Description
Automation	The system uses automated methods to shorten task duration when vulnerabilities are found.
Accuracy	Extensive dataset learning through this technology increases detection effectiveness.
Contextual Recommendations	The system generates practical data to aid developers in fixing code.
Real-time Monitoring	The system provides permanent assessment capabilities for analyzing code security.
Adaptability	The system shows competency to work with various programming languages and frameworks.

Table 2: *Gen AI benefits from detecting code vulnerabilities and assessing risks during code vulnerability scanning operations [51]*

In regards to buffer overflows, there has been a limited number of studies that explore the use of AI for detection and mitigation of memory-writing exploitation. A relevant study is *"Using Artificial Intelligence in the Context of Buffer Overflow Vulnerabilities"* [52], which investigates the detection of buffer overflow vulnerabilities using the You Only Look Once (YOLO) neural network, initially designed for object detection in computer vision. Their proposed method achieved 94.3% precision and an F1-score of 93.0% on benchmark datasets, exceeding the performance of classical static security analysis tools. A survey by *Kanaan et al.* [53] explores recent AI techniques developed for buffer overflow detection and mitigation, concluding that the recent increase in detection accuracy shows an evolution, but still requires innovation in cybersecurity for improved results.

All these studies use different AI methods and seem to generate promising results, but also show that the AI era is still at the very beginning and that, with the evolution of model quality and technologies, very promising results can be expected. However, full reliance on AI to do all the work and be 100% correct will hardly ever be possible, because hallucinations and errors occur more often than users would like them to.

4.2 Limitations and Risks of Gen-AI Integration

Achieving perfection, regardless of the subject, is considered extremely difficult and even impossible in certain cases; the same goal can be set for AI models, where perfection could correspond to 100% accuracy in answers returned to the user. This objective of perfection is currently considered not possible due to technical and human limitations.

Models can, in some cases, "hallucinate" and return inaccurate and misleading reports. The paper titled *"Why Language Models Hallucinate"*, released in an OpenAI and Georgia Tech collaboration *(Kalai et al. (2025))* [54], illustrates this phenomenon and concludes that hallucinations can be due to penalization of abstention and uncertainty in model training. This means that a model, when trained,

is less likely to be rewarded for a correct "I don't know" answer than an incorrect answer due to reinforcement learning configuration errors, forcing a mapping regardless of the answer. Human prompting is far from perfect and can also lead to unexpected results when unclear. *Zamfirescu-Pereira et al. (2023)* [55] studied how humans approached prompting and described that most errors were due to a misunderstanding of how Large Language Models (LLMs) behaved and processed prompts, while also overgeneralising some prompts and letting the LLM infer a missing context. This led to what the prompter viewed as poor performance due to their lack of understanding.

Security risks and ethical concerns are also part of the equation when it comes to using AI in a cybersecurity context. There is a very high potential for misuse of Gen-AI for automated exploit generation or reverse engineering when studying buffer overflows. One major concern among companies integrating Gen-AI into their pipelines is data leakage; it requires minimal effort for a employee to paste a confidential document into a Gen-AI model, asking it to sort or summarize the content to gain time. By doing so, inn certain cases the model may use the user provided data to train itself further and might reuse this document in future prompts if it finds it adequate creating risks of unintended disclosure. *(Gupta et al., 2023)* [56]. This happened to Samsung when an employee input confidential data into ChatGPT [57]. Ethical concerns also arise when using AI-generated code: has it been sufficiently validated by a human? Who can be considered accountable for the written code and vulnerability disclosure? A chapter of the book *Next-Generation Cybersecurity: AI, ML, and Blockchain* is dedicated to studying ethical concerns that arise from AI in cybersecurity [58].

5 Project Specifications and Research Gap

A clear research gap has been identified regarding the specific effectiveness and use of Gen-AI in mitigating and protecting against buffer overflows. This subject remains unexplored, and questions remain regarding if efficiency, accuracy and usefulness for protection against always-evolving attack techniques and behaviors are possible. Interrogations on whether Gen-AI can be more effective than other already explored techniques (CNN, GNN ...) persist, and possible integration in an industrial environment pipeline for safeguarding public software is still to be proven.

5.1 Aims and Objectives

Our project will:

- Explore the possible integration of Gen-AI tools to improve detection and mitigation of flawed systems and comparing accuracy to other existing methods
- Attempt to replicate CVE-2025-6660 heap based buffer overflow, a stack based buffer overflow and a integer buffer overflows to observe if Gen-AI could have helped in early detection of this types of vulnerabilities.
- Attempt to bypass protection mechanisms (ASLR, SSP and DEP) and analysis tools to observe if Gen-AI could still detect and patch the codebase.

.

5.2 Methodology

We will start by evaluating the assessment criteria to define a performance benchmark for models, then we will proceed in choosing the best Gen-AI models and prompt methodology to generate the most optimized results based on the same input data. Once all this is ready, we will design intentionally vulnerable code to replicate CVE-2025-6660 heap-based buffer overflow, a stack-based buffer overflow, and an integer buffer overflow to observe if Gen-AI could have improved the detection rate,

patched these vulnerabilities, and bettered an enterprise secure coding pipeline for a limited cost. Before finalizing the report, findings, and pulling conclusions from the obtained result, we will compare the use of Gen-AI with other methods to see if there exist better ways to mitigate buffer overflow risks.

There have been not identified issues that would need prolonged ethical consideration and the risk related to the testing of the vulnerable code will be mitigated by keeping everything in a controlled and secure environment.

5.3 Deliverables and Timeline

Our deliverables for this project will be: a PDF report discussing the methodology and evaluating the results obtained, the developed vulnerable source code replicating buffer overflows and Gen-AI patched source code, the exact prompts and models used to attain our results if not available publicly or fine tuned specifically.

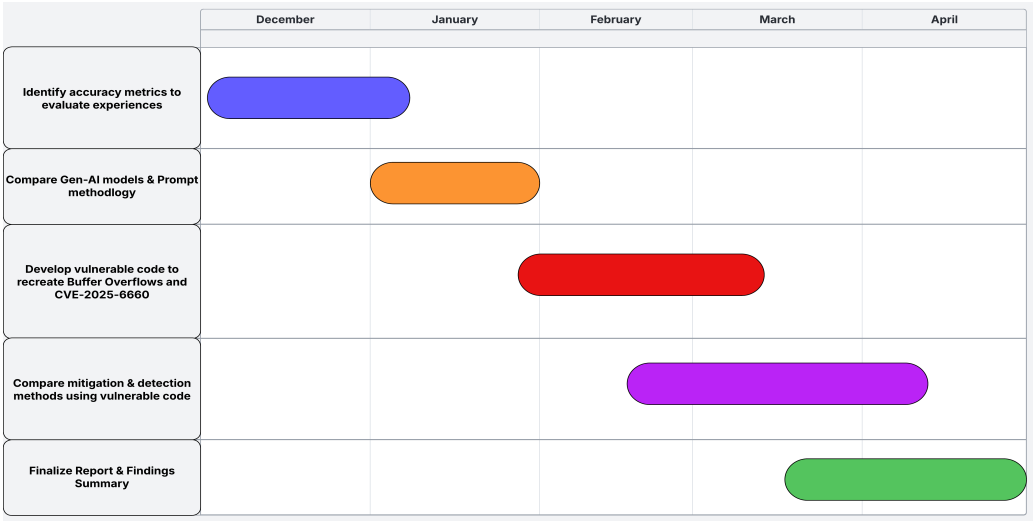

Figure 2: *Gantt chart of planned schedule for Dissertation project*

6 Conclusion

Buffer overflows are still considered a persistent threat five decades after their initial documentation, maintained by bad programming habits, legacy codebases, unsafe programming languages and the growing existence of IoT and embedded systems, which are often more inclined to be vulnerable due to their nature. Although defence mechanisms have been put in place and generally adopted, ASLR, DEP and SSP are not the perfect protection mechanisms, and always-evolving exploitation techniques are able to bypass them. Likewise, detection and mitigation using static and dynamic analysis tools are a good base to try to improve and find vulnerable code, but being generalised tools for all vulnerabilities, they still incur a lot of false positives, major overhead requirements and partial coverage for buffer overflows.

Recent advances in artificial intelligence have facilitated innovation and demonstrated promising improvements regarding this matter. Even if they seem to perform better than traditional techniques, the field is still immature, and there are still a lot of improvements to achieve to have a viable long-term solution. Their scope is often very narrow, and there is yet to be a way to adapt to new exploitation techniques in real time without redesigning the whole model from scratch. Gen-AI potential remains crucially unexplored in regard to buffer overflow mitigation and detection in real-world scenarios, constituting an opportunity for future work.

References

[1] IBM, "What is cybersecurity?" https://www.ibm.com/think/topics/cybersecurity, accessed: September 2025.

[2] W. S. Admass, Y. Y. Munaye, and A. A. Diro, "Cyber security: State of the art, challenges and future directions," *Cyber Security and Applications*, vol. 2, p. 100031, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2772918423000188

[3] J. Jacobs, S. Romanosky, I. Adjerid, and W. Baker, "Improving vulnerability remediation through better exploit prediction," *Journal of Cybersecurity*, vol. 6, no. 1, p. tyaa015, 09 2020. [Online]. Available: https://doi.org/10.1093/cybsec/tyaa015

[4] K. M. de Nobrega, A.-F. Rutkowski, and C. Saunders, "The whole of cyber defense: Syncing practice and theory," *The Journal of Strategic Information Systems*, vol. 33, no. 4, p. 101861, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S096386872400043X

[5] Fortinet, "Buffer overflow — cyber glossary," https://www.fortinet.com/uk/resources/cyberglossary/buffer-overflow#: :text=Also%20known%20as%20a%20buffer,the%20data%20in%20those%20locations., n.d., accessed: October 2025.

[6] J. P. Anderson, "Computer security technology planning study," Tech. Rep., 1972.

[7] O. Foundation, "Buffer overflow," https://owasp.org/www-community/vulnerabilities/Buffer_Overflow, n.d., accessed: October 2025.

[8] The MITRE Corporation, "2024 cwe top 25 most dangerous software weaknesses," https://cwe.mitre.org/top25/archive/2024/2024_cwe_top25.html, 2024, accessed: September 2025.

[9] A. Hikmawati and N. K. Mohammad, "Enhancing critical thinking with gen ai: A literature review," *Buletin Edukasi Indonesia*, vol. 4, no. 01, p. 40–46, Mar. 2025. [Online]. Available: https://journal.iistr.org/index.php/BEI/article/view/764

[10] C. Stryker and M. Scapicchio, "What is generative ai?" https://www.ibm.com/think/topics/generative-ai, n.d., accessed: September 2025.

[11] H. Orman, "The morris worm: a fifteen-year perspective," *IEEE Security Privacy*, vol. 1, no. 5, pp. 35–43, 2003.

[12] D. Moore, C. Shannon, and K. Claffy, "Code-red: a case study on the spread and victims of an internet worm," in *Proceedings of the 2nd ACM SIGCOMM Workshop on Internet measurment*, 2002, pp. 273–284.

[13] Cloudflare, "What is buffer overflow?" https://www.cloudflare.com/learning/security/threats/buffer-overflow/, n.d., accessed: October 2025.

[14] A. Pratama and F. A. Rafrastara, "Computer worm classification," *International Journal of Computer Science and Information Security*, vol. 10, no. 4, p. 21, 2012.

[15] M. Vallentin, "On the evolution of buffer overflows," *Munich, May*, 2007.

[16] L. Ferres, "Memory management in c: The heap and the stack," *Universidad de Concepcion*, 2010.

[17] L. Shinder and M. Cross, "Chapter 11 - passwords, vulnerabilities, and exploits," in *Scene of the Cybercrime (Second Edition)*, second edition ed., L. Shinder and M. Cross, Eds. Burlington: Syngress, 2008, pp. 467–503. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B978159749276800011X

[18] Ștefan Nicula and R. D. Zota, "Exploiting stack-based buffer overflow using modern day techniques," *Procedia Computer Science*, vol. 160, pp. 9–14, 2019, the 10th International Conference on Emerging Ubiquitous Systems and Pervasive Networks (EUSPN-2019) / The 9th International Conference on Current and Future Trends of Information and Communication Technologies in Healthcare (ICTH-2019) / Affiliated Workshops. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050919316527

[19] G. Project, "Gdb — gnu debugger (savannah)," https://savannah-checkouts.gnu.org/gnu/gdb/index.html, n.d., accessed: October 2025.

[20] National Security Agency, "Ghidra: Software reverse engineering framework," https://github.com/NationalSecurityAgency/ghidra, n.d., accessed: September 2025.

[21] C. Top and M. Dangerous, "Software weaknesses, 2020," 25.

[22] F. Meneghello, M. Calore, D. Zucchetto, M. Polese, and A. Zanella, "Iot: Internet of threats? a survey of practical security vulnerabilities in real iot devices," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 8182–8201, 2019.

[23] G. Mullen and L. Meany, "Assessment of buffer overflow based attacks on an iot operating system," in *2019 Global IoT Summit (GIoTS)*, 2019, pp. 1–6.

[24] N. V. Database, "Cve-2025-6660 detail," https://nvd.nist.gov/vuln/detail/CVE-2025-6660, 2025, published: June 25, 2025. Accessed: October 2025.

[25] ——, "Cve-2025-6191 detail," https://nvd.nist.gov/vuln/detail/CVE-2025-6191, 2025, published: June, 2025. Accessed: October 2025.

[26] C. Cowan, C. Pu, D. Maier, J. Walpole, P. Bakke, S. Beattie, A. Grier, P. Wagle, Q. Zhang, and H. Hinton, "Stackguard: Automatic adaptive detection and prevention of buffer-overflow attacks." in *USENIX security symposium*, vol. 98. San Antonio, TX, 1998, pp. 63–78.

[27] S. P. Serge Guelton, "Red hat - use compiler flags: Stack protection in gcc and clang," https://developers.redhat.com/articles/2022/06/02/use-compiler-flags-stack-protection-gcc-and-clangcontrol_flow_integrity, 2022, accessed: October 2025.

[28] B. Bierbaumer, J. Kirsch, T. Kittel, A. Francillon, and A. Zarras, "Smashing the stack protector for fun and profit," in *ICT Systems Security and Privacy Protection*, L. J. Janczewski and M. Kutyłowski, Eds. Cham: Springer International Publishing, 2018, pp. 293–306.

[29] Q. Michaud, Y. Pipereau, O. Levillain, and D. Ayed, "Robust stack smashing protection for webassembly," in *2024 IEEE Future Networks World Forum (FNWF)*. IEEE, 2024, pp. 861–866.

[30] H. Shacham, M. Page, B. Pfaff, E.-J. Goh, N. Modadugu, and D. Boneh, "On the effectiveness of address-space randomization," in *Proceedings of the 11th ACM Conference on Computer and Communications Security*, ser. CCS '04. New York, NY, USA: Association for Computing Machinery, 2004, p. 298–307. [Online]. Available: https://doi.org/10.1145/1030083.1030124

[31] H. Marco-Gisbert and I. Ripoll, "On the effectiveness of full-aslr on 64-bit linux," in *Proceedings of the In-Depth Security Conference*, 2014.

[32] N. Stojanovski, M. Gusev, D. Gligoroski, and S. Knapskog, "Bypassing data execution prevention on microsoft windows xp sp2," in *The Second International Conference on Availability, Reliability and Security (ARES'07)*, 2007, pp. 1222–1226.

[33] B. G. Cyber, "The role of non-executable stacks in preventing buffer overflows," https://bluegoatcyber.com/blog/the-role-of-non-executable-stacks-in-preventing-buffer-overflows/, n.d., accessed: October 2025.

[34] R. P. Developers, *Meet Safe and Unsafe*, n.d., accessed: October 2025.

[35] B. Hua, W. Ouyang, C. Jiang, Q. Fan, and Z. Pan, "Rupair: Towards automatic buffer overflow detection and rectification for rust," in *Proceedings of the 37th Annual Computer Security Applications Conference*, ser. ACSAC '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 812–823. [Online]. Available: https://doi.org/10.1145/3485832.3485841

[36] W. Bugden and A. Alahmar, "Rust: The programming language for safety and performance," 2022. [Online]. Available: https://arxiv.org/abs/2206.05503

[37] M. Fang and M. Hafiz, "Discovering buffer overflow vulnerabilities in the wild: an empirical study," in *Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, ser. ESEM '14. New York, NY, USA: Association for Computing Machinery, 2014. [Online]. Available: https://doi.org/10.1145/2652524.2652533

[38] M. Zitser, R. Lippmann, and T. Leek, "Testing static analysis tools using exploitable buffer overflows from open source code," in *Proceedings of the 12th ACM SIGSOFT twelfth international symposium on Foundations of software engineering*, 2004, pp. 97–106.

[39] O. Ruwase and M. S. Lam, "A practical dynamic buffer overflow detector." in *NDSS*, vol. 2004, 2004, pp. 159–169.

[40] K. Kratkiewicz and R. Lippmann, "A taxonomy of buffer overflows for evaluating static and dynamic software testing tools," in *Proceedings of Workshop on Software Security Assurance Tools, Techniques, and Metrics*, vol. 500, 2006, pp. 44–51.

[41] LetsDefend, "Top 7 reverse engineering tools," https://letsdefend.io/blog/top-7-reverse-engineering-tools, n.d., accessed: October 2025.

[42] W. D. Heaven, "The inside story of how chatgpt was built from the people who made it," *MIT Technology Review*, Mar 2023, accessed: October 2025.

[43] Turing, "The 24 best generative ai tools in 2025," https://www.turing.com/resources/generative-ai-tools, 2024, accessed: October 2025.

[44] A. B. Yadav, "Gen ai-driven electronics: innovations, challenges and future prospects," in *International congress on models and methods in modern investigations*, 2023, pp. 113–121.

[45] R. Mortlock and C. Lucas, "Generative artificial intelligence (gen-ai) in pharmacy education: Utilization and implications for academic integrity: A scoping review," *Exploratory Research in Clinical and Social Pharmacy*, vol. 15, p. 100481, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2667276624000787

[46] Microsoft, "Generative ai vs. other types of ai," https://www.microsoft.com/en-us/ai/ai-101/generative-ai-vs-other-types-of-ai, n.d., accessed: October 2025.

[47] S. Rajapaksha, J. Senanayake, H. Kalutarage, and M. O. Al-Kadri, "Ai-powered vulnerability detection for secure source code development," in *Innovative Security Solutions for Information Technology and Communications*, G. Bella, M. Doinea, and H. Janicke, Eds. Cham: Springer Nature Switzerland, 2023, pp. 275–288.

[48] A. Amarasinghe, W. Wijesinghe, D. Nirmana, A. Jayakody, and A. Priyankara, "Ai based cyber threats and vulnerability detection, prevention and prediction system," in *2019 International Conference on Advancements in Computing (ICAC)*, 2019, pp. 363–368.

[49] A. Schreiber and I. Schreiber, "Removing unknowns: Gen-ai as a cybersecurity interactive advisor for human experts," *Information and Computer Security*, 09 2025. [Online]. Available: https://doi.org/10.1108/ICS-03-2025-0093

[50] A. Zewe, "Explained: Generative ai," *MIT News*, Nov 2023, accessed: October 2025.

[51] S. Shukla, "Gen ai for code vulnerability detection and risk analysis," *Available at SSRN 5173531*, 2023.

[52] O. Savenko, Y. Sierhieiev, P. Gaj, and J. Balej, "Using artificial intelligence in the context of buffer overflow vulnerabilities," 2025.

[53] E. Kanaan, S. S. Alam, and M. S. Akter, "Survey of machine learning techniques for detecting buffer overflow vulnerabilities."

[54] A. T. Kalai, O. Nachum, S. S. Vempala, and E. Zhang, "Why language models hallucinate," *arXiv preprint arXiv:2509.04664*, 2025.

[55] J. D. Zamfirescu-Pereira, R. Y. Wong, B. Hartmann, and Q. Yang, "Why johnny can't prompt: how non-ai experts try (and fail) to design llm prompts," in *Proceedings of the 2023 CHI conference on human factors in computing systems*, 2023, pp. 1–21.

[56] M. Gupta, C. Akiri, K. Aryal, E. Parker, and L. Praharaj, "From chatgpt to threatgpt: Impact of generative ai in cybersecurity and privacy," *IEEE Access*, vol. 11, pp. 80 218–80 245, 2023.

[57] L. Maddison, "Samsung workers made a major error by using chatgpt," *TechRadar*, Apr 2023, accessed: October 2025.

[58] K. Kaushik, A. Khan, A. Kumari, I. Sharma, and R. Dubey, *Ethical Considerations in AI-Based Cybersecurity*. Singapore: Springer Nature Singapore, 2024, pp. 437–470. [Online]. Available: https://doi.org/10.1007/978-981-97-1249-6$_1$9